In [1]:
```
import os
import sys

notebook_dir = os.getcwd()
project_root_path = os.path.dirname(notebook_dir)
sys.path.insert(0, project_root_path)

from src import ModelXtoC
from scripts.run_preprocessing import preprocessing_main
from src.utils import *
from src.training import run_epoch_x_to_c
```

In [3]:
```
concept_labels, train_loader, test_loader = preprocessing_main(trim_conce
```

```
Found 11788 images.
Processing in 369 batches of size 32 (for progress reporting)...
Processing batches: 100%|██████████████████| 369/369 [00:59<00:00,  6.2
5it/s]
Finished processing.
Successfully transformed: 11788 images.
Found 11788 unique images.
Found 312 unique concepts.
Generated concept matrix with shape: (11788, 312)
Found 200 classes.
Found labels for 11788 images.
Generated one-hot matrix with shape: (11788, 200)
Split complete: 5994 train images, 5794 test images.
Dataset initialized with 5994 pre-sorted items.
Dataset initialized with 5794 pre-sorted items.
```

Training Implementation

In [5]:
```
from src.utils import find_class_imbalance
from config import N_CONCEPTS, N_CLASSES
import torch
import torch.nn as nn
import torch.optim as optim
```

In [4]:
```
# N_CONCEPTS = 109
```

Find device to run model on (CPU or GPU).

In [9]:
```
device = torch.device("cuda" if torch.cuda.is_available()
                      else "mps" if torch.backends.mps.is_available()
                      else "cpu")
print(f"Using device: {device}")
```

```
Using device: mps
```

Instantiate the model.

In [6]:
```
model = ModelXtoC(pretrained=True,
                  freeze=True,
                  n_classes=N_CLASSES, # Still needed for model structure,
                  use_aux=True,
```

```
                 n_concepts=N_CONCEPTS)

model = model.to(device)
print("Model Instantiated (X -> C)")
```

```
Model Instantiated (X -> C)
```

Loss

We use weighted loss.

`BCEWithLogitsLoss()` performs 2 steps:

1. $\sigma(x)$
   - Applies the sigmoid function to the logits to get probabilities.
2. $\mathrm{BCE}(\sigma(x), y) = y \cdot \log(\sigma(x)) + (1 - y) \cdot (1 - \log(\sigma(x)))$
   - Compute binary cross-entropy between output probabilities ($\sigma(x)$) and ground truths (y)

In [10]:
```
use_weighted_loss = True # Set to False for simple unweighted loss

if use_weighted_loss:
    concept_weights = find_class_imbalance(concept_labels)
    attr_criterion = [nn.BCEWithLogitsLoss(weight=torch.tensor([ratio], d
                      for ratio in concept_weights]
else:
    attr_criterion = [nn.BCEWithLogitsLoss() for _ in range(N_CONCEPTS)]
```

Optimiser

Use same settings as used in CBM repo.

In [11]:
```
lr = 0.01
weight_decay = 0.00004 # same as lambda in L2-regularisation

optimizer = optim.SGD(filter(lambda p: p.requires_grad, model.parameters(
                      lr=lr,
                      momentum=0.9,
                      weight_decay=weight_decay)

# scheduler_step = n -> decrease the LR every n epochs
scheduler_step = 1000
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=scheduler_step

print("Optimizer and Scheduler Ready")
```

```
Optimizer and Scheduler Ready
```

Training and Validation Loops

In [12]:
```
epochs = 50
log_interval = 50

best_val_acc = 0.0
```

In [ ]:
```
for epoch in range(epochs):
    print(f"--- Epoch {epoch+1}/{epochs} ---")

    # Train
    train_loss, train_acc = run_epoch_x_to_c(model, train_loader, attr_cr
    print(f'Epoch {epoch+1} Train Summary | Loss: {train_loss:.4f} | Acc:

    # Validate
    if test_loader:
        with torch.no_grad():
            val_loss, val_acc = run_epoch_x_to_c(model, test_loader, attr

        print(f'Epoch {epoch+1} Val Summary   | Loss: {val_loss:.4f} | Ac

        # Save best model based on validation accuracy
        if val_acc > best_val_acc:
            print(f"Validation accuracy improved ({best_val_acc:.3f} -> {
            best_val_acc = val_acc
            torch.save(model.state_dict(), 'x_to_c_best_model.pth')
            print("Model saved to x_to_c_best_model.pth")

    # Scheduler step
    scheduler.step()
    print(f"Current LR: {optimizer.param_groups[0]['lr']}")
```

LOAD BEST MODEL FROM CBM PAPER

In [13]:
```
from config import PROJECT_ROOT

best_model = os.path.join(PROJECT_ROOT, 'models', 'best_model_1.pth')
model = torch.load(best_model, map_location=device, weights_only=False)
print("Best model loaded.")

N_CONCEPTS=112

if test_loader:
    with torch.no_grad():
        test_loss, test_acc = run_epoch_x_to_c(model, test_loader, attr_c

print(f'Best Model Summary   | Loss: {test_loss:.4f} | Acc: {test_acc:.3f
```

```
Best model loaded.
Best Model Summary   | Loss: 50.8061 | Acc: 80.848
```

In [15]:
```
best_model = os.path.join(PROJECT_ROOT, 'models', 'best_model_2.pth')
model = torch.load(best_model, map_location=device, weights_only=False)
print("Best model loaded.")

if test_loader:
    with torch.no_grad():
        test_loss, test_acc = run_epoch_x_to_c(model, test_loader, attr_c

print(f'Best Model Summary   | Loss: {test_loss:.4f} | Acc: {test_acc:.3f
```

```
Best model loaded.
Best Model Summary   | Loss: 52.0604 | Acc: 80.770
```

In [14]:
```
best_model = os.path.join(PROJECT_ROOT, 'models', 'best_model_3.pth')
model = torch.load(best_model, map_location=device, weights_only=False)
print("Best model loaded.")

if test_loader:
    with torch.no_grad():
        test_loss, test_acc = run_epoch_x_to_c(model, test_loader, attr_c

print(f'Best Model Summary   | Loss: {test_loss:.4f} | Acc: {test_acc:.3f
```

```
Best model loaded.
Best Model Summary   | Loss: 51.3975 | Acc: 80.806
```

In [ ]:

In [ ]:

In [ ]: